Status	Finished
Started	Thursday, 29 October 2026, 6:12 PM
Completed	Thursday, 29 October 2026, 6:29 PM
Duration	16 mins 46 secs
Grade	**6.00** out of 10.00 (**60**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    lines = sys.stdin.read().splitlines()
    if not lines:
        return

    # Process inputs in pairs (Tuple line, then K line)
    iterator = iter(lines)
    for tuple_line in iterator:
        try:
            k_line = next(iterator)

            # Convert string input "5,6,5,7,7,8" into a tuple of integers
            t = tuple(map(int, tuple_line.split(',')))
            K = int(k_line)

            # Logic
            seen = set()
            distinct_pairs = set()
            for num in t:
                target = K - num
                if target in seen:
                    distinct_pairs.add(tuple(sorted((num, target))))
                seen.add(num)

            print(len(distinct_pairs))

        except StopIteration:
            break

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓

	Input	Expected	Got	
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
def check_subset():
    # Read three lines of input, splitting each line into a set of words
    try:
        set1 = set(input().split())
        set2 = set(input().split())
        set3 = set(input().split())
    except EOFError:
        return

    # Check if set3 is a subset of both set1 and set2
    if set3.issubset(set1) and set3.issubset(set2):
        print("yes")
        print("set3 is subset of set1 and set2")
    else:
        print("No")

# Execute the function
check_subset()

```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
s = input()

if s and set(s) <= {'0', '1'}:
    print("Yes")
else:
    print("No")
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

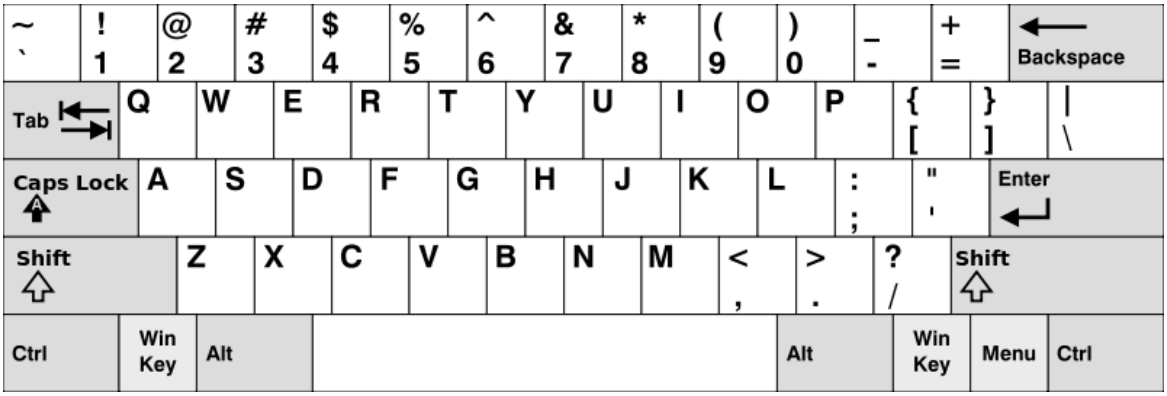
Incorrect

Mark 0.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below.*

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
class Solution:
    def findWords(self, words: list[str]) -> list[str]:
        row1 = set("qwertyuiop")
        row2 = set("asdfghjkl")
        row3 = set("zxcvbnm")

        result = []

        for word in words:
            letters = set(word.lower())
            if letters <= row1 or letters <= row2 or letters <= row3:
                result.append(word)
```

```
13
14          return result
```

	Input	Expected	
✗	4 Hello Alaska Dad Peace	Alaska Dad	✗
✗	1 omk	No words	✗
✗	2 adsfd afd	adsfd afd	✗

Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **5**

Incorrect

Mark 0.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
def findDuplicate(nums: list[int]) -> int:
    seen = set()
    for num in nums:
        if num in seen:
            return num
        seen.add(num)
    return -1

# Test Case 1
print(findDuplicate([1, 3, 4, 4, 2]))       # Output: 4

# Test Case 2
print(findDuplicate([1, 2, 2, 3, 4, 5, 6, 7]))  # Output: 2

```

	Input	Expected	Got	
✗	1 3 4 4 2	4	4 2	✗
✗	1 2 2 3 4 5 6 7	2	4 2	✗

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```python
def count_operations(nums: tuple[int, ...]) -> int:
    # Track the remaining elements as an immutable tuple
    current_tuple = nums

    # Pre-sort the elements to always know the current smallest value in O(1)
    # Using sorted() internally creates a unique, ordered sequence from the elements
    sorted_elements = sorted(set(nums))
    min_index = 0

    operations = 0

    while current_tuple:
        operations += 1
        first = current_tuple[0]
        current_min = sorted_elements[min_index]

        if first == current_min:
            # Remove the first element by slicing
            current_tuple = current_tuple[1:]
            # The next smallest element is now active
            min_index += 1
        else:
            # Move the first element to the end
            current_tuple = current_tuple[1:] + (first,)

    return operations

```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all input from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Line 1 contains the sizes of the two arrays (not strictly needed for Python parsing)
    sizes = list(map(int, input_data[0].split()))

    # Line 2 and Line 3 contain the two arrays
    array1 = list(map(int, input_data[1].split()))
    array2 = list(map(int, input_data[2].split()))

    # Convert lists to sets for efficient mathematical set operations
    set1 = set(array1)
    set2 = set(array2)

    # Elements in set1 but not in set2, and elements in set2 but not in set1
    # This automatically eliminates all common elements
    unique_to_1 = set1 - set2
    unique_to_2 = set2 - set1

    # Gather non-repeating elements while preserving the original input order
```

```
    result = []
    for num in array1:
        if num in unique_to_1 and num not in result:
            result.append(num)

    for num in array2:
        if num in unique_to_2 and num not in result:
            result.append(num)

    # Output processing based on whether unique elements exist
    if not result:
        print("NO SUCH ELEMENTS")
    else:
        # Print elements separated by a space
        print(*(result))
        # Print the total count of non-repeating elements
        print(len(result))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Incorrect

Mark 0.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input
    input_lines = sys.stdin.read().splitlines()
    if len(input_lines) < 2:
        return

    # First line is the text, second line contains the broken letters
    text = input_lines[0]
    brokenLetters = input_lines[1]

    # Convert broken letters to a set for O(1) lookups
    broken_set = set(brokenLetters)

    # Split the text into individual words
    words = text.split()

    can_type_count = 0

    for word in words:
        # If any character in the word is broken, we skip it
        if any(char in broken_set for char in word):
            continue
        can_type_count += 1

    print(can_type_count)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓
✗	Welcome to REC e	1	2	✗

	Input	Expected	Got	
✓	`Faculty Upskilling in Python Programming` `ak`	2	2	✓

Your code must pass all tests to earn any marks. Try again.

Show differences

Incorrect

Marks for this submission: 0.00/1.00.

Question **9**

Incorrect

Mark 0.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the **`10`-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```
class Solution:
    def findRepeatedDnaSequences(self, s: str) -> list[str]:
        seen = set()
        repeated = set()

        for i in range(len(s) - 9):
            seq = s[i:i + 10]
            if seq in seen:
                repeated.add(seq)
            else:
                seen.add(seq)

        return list(repeated)
```

	Input	Expected	
✗	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA	✗
✗	AAAAAAAAAAAAA	AAAAAAAAAA	✗

Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read all lines from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # The first token is the array length n
    n = int(input_data[0])

    # The remaining tokens are the elements of the array
    array_elements = input_data[1:n+1]

    # Use dict.fromkeys to remove duplicates while preserving input order
    distinct_elements = list(dict.fromkeys(array_elements))

    # Print the distinct elements separated by a single space
    print(*(distinct_elements))

if __name__ == "__main__":
    main()

```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.